Emergency Rescue Priority System

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

struct EmerReq {
    char req_id[50];
    char location[50];
    int no_of_people;
    char emglvl[50];
    char team[50];
    char resource[50];
    char status[50];
    int priority;
};

enum rescueteam  { team1, team2, team3, team4 };
enum priority { low = 1, medium, critical };

const char *teamNames[4] = {"Team1", "Team2", "Team3", "Team4"};
struct EmerReq records[100];
int teamBusy[4] = {0};
int count = 0;

int getPriority(char level[]) {
    if (strcmp(level, "critical") == 0)
        return critical;
     if (strcmp(level, "medium") == 0)
        return medium;
       if (strcmp(level, "low") == 0)
        return low;
    return 0;
}

int findRequest(char id[]) {
    for (int i = 0; i < count; i++) {
        if (strcmp(records[i].req_id, id) == 0)
            return i;
    }
    return -1;
}

void registerRequest() {
    if (count >= 100) {
```

```
        printf("Request limit reached!\n");
        return;
    }

    struct EmerReq r;
    printf("Enter Request ID: ");
    scanf("%49s", r.req_id);

    if (findRequest(r.req_id) != -1) {
        printf("Request ID already exists!\n");
        return;
    }

    printf("Enter Location: ");
    scanf(" %49[^\n]", r.location);
    printf("Enter Number of People: ");
    scanf("%d", &r.no_of_people);

    if (r.no_of_people <= 0) {
        printf("Invalid number of people!\n");
        return;
    }

    printf("Enter Emergency Level (critical/medium/low): ");
    scanf("%49s", r.emglvl);
    r.priority = getPriority(r.emglvl);

    if (r.priority == 0) {
        printf("Invalid emergency level!\n");
        return;
    }

    printf("Enter Required Resource: ");
    scanf(" %49[^\n]", r.resource);

    strcpy(r.team, "Unassigned");
    strcpy(r.status, "Pending");
    records[count] = r;
    count++;
    printf("Emergency request registered successfully!\n");
}

void sortRequests() {
    for (int i = 0; i < count - 1; i++) {
        for (int j = i + 1; j < count; j++) {
            if (records[i].priority < records[j].priority) {
                struct EmerReq temp = records[i];
                records[i] = records[j];
```

```
                records[j] = temp;
            }
        }
    }
}

void displayRequests() {
    if (count == 0) {
        printf("No emergency requests available!\n");
        return;
    }

    sortRequests();
    printf("\nRequests Sorted by Priority\n");

    for (int i = 0; i < count; i++) {
        printf("\nRequest ID: %s\n", records[i].req_id);
        printf("Location: %s\n", records[i].location);
        printf("People: %d\n", records[i].no_of_people);
        printf("Emergency: %s\n", records[i].emglvl);
        printf("Priority: %d\n", records[i].priority);
        printf("Resource: %s\n", records[i].resource);
        printf("Team: %s\n", records[i].team);
        printf("Status: %s\n", records[i].status);
    }
}

void assignTeam() {
    char id[50];
    printf("Enter Request ID: ");
    scanf("%49s", id);

    int found = findRequest(id);
    if (found == -1) {
        printf("Request not found!\n");
        return;
    }

    if (strcmp(records[found].status, "Completed") == 0) {
        printf("Request is already completed!\n");
        return;
    }

    if (strcmp(records[found].team, "Unassigned") != 0) {
        printf("A team is already assigned: %s\n", records[found].team);
        return;
    }
```

```
    for (int i = 0; i < 4; i++) {
        if (teamBusy[i] == 0) {
            teamBusy[i] = 1;
            strcpy(records[found].team, teamNames[i]);
            strcpy(records[found].status, "InProgress");
            printf("%s assigned to request %s\n", teamNames[i], records[found].req_id);
            return;
        }
    }

    printf("No rescue team is currently available!\n");
}

void updateStatus() {
    char id[50], status[50];
    printf("Enter Request ID: ");
    scanf("%49s", id);

    int found = findRequest(id);
    if (found == -1) {
        printf("Request not found!\n");
        return;
    }

    printf("Enter new status (Pending/InProgress/Completed): ");
    scanf("%49s", status);

    if (strcmp(status, "Pending") != 0 &&
        strcmp(status, "InProgress") != 0 &&
             strcmp(status, "Completed") != 0) {
            printf("Invalid status!\n");
        return;
    }

        if (strcmp(records[found].status, "Completed") == 0) {
                printf("Completed requests cannot be updated!\n");
                return;
    }

       if (strcmp(status, "InProgress") == 0 &&
            strcmp(records[found].team, "Unassigned") == 0) {
              printf("Assign a rescue team before starting the request!\n");
              return;
    }

    if (strcmp(status, "Completed") == 0 &&
        strcmp(records[found].team, "Unassigned") != 0) {
        for (int i = 0; i < 4; i++) {
```

```
            if (strcmp(records[found].team, teamNames[i]) == 0)
                teamBusy[i] = 0;
        }
    }

    strcpy(records[found].status, status);
    printf("Request status updated successfully!\n");
}

void searchRequest() {
    char id[50];
    printf("Enter Request ID: ");
    scanf("%49s", id);

    int found = findRequest(id);
    if (found == -1) {
        printf("Request not found!\n");
        return;
    }

      printf("\nRequest ID: %s\n", records[found].req_id);
     printf("Location: %s\n", records[found].location);
       printf("People: %d\n", records[found].no_of_people);
        printf("Emergency: %s\n", records[found].emglvl);
         printf("Priority: %d\n", records[found].priority);
        printf("Resource: %s\n", records[found].resource);
      printf("Team: %s\n", records[found].team);
     printf("Status: %s\n", records[found].status);
}

void emergencySummary() {
    int criticalCount = 0, mediumCount = 0, lowCount = 0;
    int pending = 0, inProgress = 0, completed = 0, people = 0;

    for (int i = 0; i < count; i++) {
        people += records[i].no_of_people;

        if (records[i].priority == critical)
            criticalCount++;
        else if (records[i].priority == medium)
            mediumCount++;
        else if (records[i].priority == low)
            lowCount++;

        if (strcmp(records[i].status, "Pending") == 0)
            pending++;
        else if (strcmp(records[i].status, "InProgress") == 0)
            inProgress++;
```

```
        else if (strcmp(records[i].status, "Completed") == 0)
            completed++;
    }

        printf("\nEmergency Summary\n");
     printf("Total Requests: %d\n", count);
        printf("Total People Affected: %d\n", people);
       printf("Critical Requests: %d\n", criticalCount);
      printf("Medium Requests: %d\n", mediumCount);
    printf("Low Requests: %d\n", lowCount);
     printf("Pending Requests: %d\n", pending);
      printf("In Progress Requests: %d\n", inProgress);
       printf("Completed Requests: %d\n", completed);
}

int main() {
    int choice;

    while (1) {
        printf("\n===== Emergency Rescue Management =====\n");
        printf("1. Register Emergency Request\n");
        printf("2. Calculate and Assign Priority\n");
        printf("3. Display Requests by Priority\n");
        printf("4. Assign Rescue Team\n");
        printf("5. Update Request Status\n");
        printf("6. Search Request\n");
        printf("7. Generate Emergency Summary\n");
        printf("8. Exit\n");
        printf("Enter your choice: ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid input!\n");
            break;
        }

        switch (choice) {
            case 1:
                registerRequest();
                break;
            case 2:
                for (int i = 0; i < count; i++)
                    records[i].priority = getPriority(records[i].emglvl);
                sortRequests();
                printf("Priorities calculated and requests sorted!\n");
                break;
            case 3:
                displayRequests();
                break;
```

```
            case 4:
                assignTeam();
                break;
            case 5:
                updateStatus();
                break;
            case 6:
                searchRequest();
                break;
            case 7:
                emergencySummary();
                break;
            case 8:
                printf("Exiting program...\n");
                return 0;
            default:
                printf("Invalid choice!\n");
        }
    }

    return 0;
}
```

Smart Campus Complaint System

```
#include <stdio.h>
#include <string.h>

struct Complaint {
    char id[50];
    char category[50];
    char problem[150];
    char location[100];
    char severity[20];
    char status[30];
    int priority;
};
```

```
enum Priority { LOW = 1, MEDIUM, HIGH, CRITICAL };

struct Complaint complaints[100];
int count = 0;

int getPriority(char severity[]) {
    if (strcmp(severity, "Critical") == 0)
        return CRITICAL;
    if (strcmp(severity, "High") == 0)
        return HIGH;
    if (strcmp(severity, "Medium") == 0)
        return MEDIUM;
    if (strcmp(severity, "Low") == 0)
        return LOW;
    return 0;
}

char *priorityName(int priority) {

    switch (priority) {

        case CRITICAL: return "Critical";

        case HIGH: return "High";

        case MEDIUM: return "Medium";

        case LOW: return "Low";
        default: return "Unknown";
    }
}

int findComplaint(char id[]) {
    for (int i = 0; i < count; i++) {
        if (strcmp(complaints[i].id, id) == 0)
            return i;
    }
    return -1;
}

void registerComplaint() {
    if (count >= 100) {
        printf("Complaint limit reached!\n");
        return;
    }

    struct Complaint c;
    printf("Enter Complaint ID: ");
```

```
    scanf("%49s", c.id);

    if (findComplaint(c.id) != -1) {
        printf("Complaint ID already exists!\n");
        return;
    }

    printf("Enter Category: ");
    scanf(" %49[^\n]", c.category);
    printf("Enter Problem Description: ");
    scanf(" %149[^\n]", c.problem);
    printf("Enter Location: ");
    scanf(" %99[^\n]", c.location);
    printf("Enter Severity (Critical/High/Medium/Low): ");
    scanf("%19s", c.severity);

    c.priority = getPriority(c.severity);
    if (c.priority == 0) {
        printf("Invalid severity!\n");
        return;
    }

    strcpy(c.status, "Pending");
    complaints[count] = c;
    count++;
    printf("Complaint registered successfully!\n");
}

void displayComplaint(struct Complaint c) {
    printf("\nComplaint ID: %s\n", c.id);
    printf("Category: %s\n", c.category);
    printf("Problem: %s\n", c.problem);
    printf("Location: %s\n", c.location);
    printf("Priority: %s\n", priorityName(c.priority));
    printf("Status: %s\n", c.status);
}

void sortComplaints() {
    int choice;
    printf("Sort by:\n");
    printf("1. Priority\n");
    printf("2. Complaint ID\n");
    printf("3. Category\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    for (int i = 0; i < count - 1; i++) {
        for (int j = i + 1; j < count; j++) {
```

```
            int swap = 0;

            if (choice == 1 && complaints[i].priority < complaints[j].priority)
                swap = 1;
            else if (choice == 2 && strcmp(complaints[i].id, complaints[j].id) > 0)
                swap = 1;
            else if (choice == 3 && strcmp(complaints[i].category, complaints[j].category) > 0)
                swap = 1;

            if (swap) {
                struct Complaint temp = complaints[i];
                complaints[i] = complaints[j];
                complaints[j] = temp;
            }
        }
    }

    printf("Complaints sorted successfully!\n");
}

void displayAllComplaints() {
    if (count == 0) {
        printf("No complaints registered!\n");
        return;
    }

    for (int i = 0; i < count; i++)
        displayComplaint(complaints[i]);
}

void searchComplaint() {
    int choice, found = 0;
    char searchValue[150];

    printf("Search by:\n");
    printf("1. Complaint ID\n");
    printf("2. Category\n");
    printf("3. Location\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
    printf("Enter search value: ");
    scanf(" %149[^\n]", searchValue);

    for (int i = 0; i < count; i++) {
        int match = 0;

        if (choice == 1 && strcmp(complaints[i].id, searchValue) == 0)
            match = 1;
```

```
        else if (choice == 2 && strcmp(complaints[i].category, searchValue) == 0)
            match = 1;
        else if (choice == 3 && strcmp(complaints[i].location, searchValue) == 0)
            match = 1;

        if (match) {
            displayComplaint(complaints[i]);
            found = 1;
        }
    }

    if (!found)
        printf("No matching complaint found!\n");
}

void updateStatus() {
    char id[50], status[30];
    printf("Enter Complaint ID: ");
    scanf("%49s", id);

    int found = findComplaint(id);
    if (found == -1) {
        printf("Complaint not found!\n");
        return;
    }

    printf("Enter new status (Pending/InProgress/Resolved): ");
    scanf("%29s", status);

    if (strcmp(status, "Pending") != 0 &&

        strcmp(status, "InProgress") != 0 &&

        strcmp(status, "Resolved") != 0) {
        printf("Invalid status!\n");
        return;
    }

    strcpy(complaints[found].status, status);

    printf("Complaint status updated successfully!\n");
}

void displayPending() {
    int found = 0;

    for (int i = 0; i < count; i++) {
        if (strcmp(complaints[i].status, "Pending") == 0) {
```

```
            displayComplaint(complaints[i]);
            found = 1;
        }
    }

    if (!found)
        printf("No pending complaints!\n");
}

void complaintSummary() {
    int critical = 0, high = 0, medium = 0, low = 0;

    int pending = 0, progress = 0, resolved = 0;

    for (int i = 0; i < count; i++) {
        switch (complaints[i].priority) {

            case CRITICAL: critical++; break;
            case HIGH: high++;
            break;

            case MEDIUM: medium++;
             break;
            case LOW: low++; break;
        }

        if (strcmp(complaints[i].status, "Pending") == 0)
            pending++;
        else if (strcmp(complaints[i].status, "InProgress") == 0)
            progress++;

        else if (strcmp(complaints[i].status, "Resolved") == 0)
            resolved++;

    }

    printf("\n===== Complaint Summary =====\n");
    printf("Total Complaints: %d\n", count);
    printf("Critical Priority: %d\n", critical);
    printf("High Priority: %d\n", high);
    printf("Medium Priority: %d\n", medium);
    printf("Low Priority: %d\n", low);
    printf("Pending: %d\n", pending);
    printf("In Progress: %d\n", progress);
    printf("Resolved: %d\n", resolved);
}

int main() {
```

```
int choice;

while (1) {
   printf("\n===== Campus Complaint Management =====\n");
    printf("1. Register Complaint\n");
     printf("2. Assign Priority\n");
      printf("3. Search Complaint\n");
        printf("4. Sort Complaints\n");
      printf("5. Update Complaint Status\n");
      printf("6. Display Pending Complaints\n");
        printf("7. Generate Complaint Summary\n");
        printf("8. Display All Complaints\n");
          printf("9. Exit\n");
         printf("Enter your choice: ");

   if (scanf("%d", &choice) != 1) {
       printf("Invalid input!\n");
       break;
   }

   switch (choice) {

       case 1:
           registerComplaint();
           Break;

       case 2:
           for (int i = 0; i < count; i++)
               complaints[i].priority = getPriority(complaints[i].severity);

           printf("Complaint priorities calculated successfully!\n");

           break;
       case 3:
           searchComplaint();
           break;
       case 4:
           sortComplaints();
           break;
       case 5:
           updateStatus();
           break;
       case 6:
           displayPending();
           break;
       case 7:
           complaintSummary();
           break;
```

```
            case 8:

                displayAllComplaints();
                break;
            case 9:

                printf("Exiting program...\n");
                return 0;
            default:

                printf("Invalid choice!\n");
        }
    }

    return 0;
}
```

Smart Hospital Emergency Queue System

```
#include <stdio.h>
#include <string.h>

struct Patient {
    char id[50];
    char name[50];
    int age;
    char condition[100];
    int priority;
    char status[30];
    char doctor[50];
    char bed[50];
};

enum Priority { CRITICAL = 1, HIGH, MEDIUM, LOW };

struct Patient patients[100];
char doctorNames[4][50] = {"Doctor1", "Doctor2", "Doctor3", "Doctor4"};
char bedNames[4][50] = {"Bed1", "Bed2", "Bed3", "Bed4"};
```

```
int doctorBusy[4] = {0}, bedBusy[4] = {0};
int count = 0;

int findPatient(char id[]) {
    for (int i = 0; i < count; i++) {
        if (strcmp(patients[i].id, id) == 0)
            return i;
    }
    return -1;
}

int getPriority(int choice) {
    if (choice >= CRITICAL && choice <= LOW)
        return choice;
    return 0;
}

char *priorityName(int priority) {
    switch (priority) {
        case CRITICAL: return "Critical";
        case HIGH: return "High";
        case MEDIUM: return "Medium";
        case LOW: return "Low";
        default: return "Unknown";
    }
}

void registerPatient() {
    if (count >= 100) {
        printf("Patient limit reached!\n");
        return;
    }

    struct Patient p;
    printf("Enter Patient ID: ");
    scanf("%49s", p.id);

    if (findPatient(p.id) != -1) {
        printf("Patient ID already exists!\n");
        return;
    }

    printf("Enter Patient Name: ");
    scanf(" %49[^\n]", p.name);
    printf("Enter Age: ");
    scanf("%d", &p.age);

    if (p.age <= 0 || p.age > 130) {
```

```
        printf("Invalid age!\n");
        return;
    }

    printf("Enter Patient Condition: ");
    scanf(" %99[^\n]", p.condition);
    printf("Enter Priority (1-Critical, 2-High, 3-Medium, 4-Low): ");
    scanf("%d", &p.priority);

    p.priority = getPriority(p.priority);
    if (p.priority == 0) {
        printf("Invalid priority!\n");
        return;
    }

    strcpy(p.status, "Waiting");
    strcpy(p.doctor, "Unassigned");
    strcpy(p.bed, "Unassigned");
    patients[count] = p;
    count++;

    printf("Patient registered successfully!\n");
}

void displayPatient(struct Patient p) {
    printf("\nPatient ID: %s\n", p.id);
    printf("Name: %s\n", p.name);
    printf("Age: %d\n", p.age);
    printf("Condition: %s\n", p.condition);
    printf("Priority: %d - %s\n", p.priority, priorityName(p.priority));
    printf("Status: %s\n", p.status);
    printf("Doctor: %s\n", p.doctor);
    printf("Bed: %s\n", p.bed);
}

void sortPatients() {
    for (int i = 0; i < count - 1; i++) {
        for (int j = i + 1; j < count; j++) {
            if (patients[i].priority > patients[j].priority) {
                struct Patient temp = patients[i];
                patients[i] = patients[j];
                patients[j] = temp;
            }
        }
    }
}

void displayPatients() {
```

```
    if (count == 0) {
        printf("No patients registered!\n");
        return;
    }

    sortPatients();
    printf("\nPatients in Priority Order\n");

    for (int i = 0; i < count; i++)
        displayPatient(patients[i]);
}

void callNextPatient() {
    sortPatients();

    for (int i = 0; i < count; i++) {
        if (strcmp(patients[i].status, "Waiting") == 0) {
            printf("\nNext Patient Called\n");
            displayPatient(patients[i]);
            return;
        }
    }

    printf("No patients are waiting!\n");
}

void assignDoctorBed() {
    char id[50];
    printf("Enter Patient ID: ");
    scanf("%49s", id);

    int found = findPatient(id);
    if (found == -1) {
        printf("Patient not found!\n");
        return;
    }

    if (strcmp(patients[found].status, "Completed") == 0) {
        printf("Treatment is already completed!\n");
        return;
    }

    int doctor = -1, bed = -1;

    if (strcmp(patients[found].doctor, "Unassigned") != 0) {
        printf("Doctor already assigned: %s\n", patients[found].doctor);
    } else {
        for (int i = 0; i < 4; i++) {
```

```
            if (doctorBusy[i] == 0) {
                doctor = i;
                break;
            }
        }

        if (doctor == -1) {
            printf("No doctor available!\n");
            return;
        }
    }

    if (strcmp(patients[found].bed, "Unassigned") != 0) {
        printf("Bed already assigned: %s\n", patients[found].bed);
    } else {
        for (int i = 0; i < 4; i++) {
            if (bedBusy[i] == 0) {
                bed = i;
                break;
            }
        }

        if (bed == -1) {
            printf("No bed available!\n");
            return;
        }
    }

    if (doctor != -1) {
        doctorBusy[doctor] = 1;
        strcpy(patients[found].doctor, doctorNames[doctor]);
    }

    if (bed != -1) {
        bedBusy[bed] = 1;
        strcpy(patients[found].bed, bedNames[bed]);
    }

    strcpy(patients[found].status, "InTreatment");
    printf("Doctor and bed assignment completed!\n");
    displayPatient(patients[found]);
}

void updateStatus() {
    char id[50], status[30];
    printf("Enter Patient ID: ");
    scanf("%49s", id);
```

```
    int found = findPatient(id);
    if (found == -1) {
        printf("Patient not found!\n");
        return;
    }

    printf("Enter Status (Waiting/InTreatment/Completed): ");
    scanf("%29s", status);

    if (strcmp(status, "Waiting") != 0 &&
        strcmp(status, "InTreatment") != 0 &&
        strcmp(status, "Completed") != 0) {
        printf("Invalid status!\n");
        return;
    }

    if (strcmp(patients[found].status, "Completed") == 0) {
        printf("Completed patient records cannot be updated!\n");
        return;
    }

    if (strcmp(status, "InTreatment") == 0 &&
        (strcmp(patients[found].doctor, "Unassigned") == 0 ||
         strcmp(patients[found].bed, "Unassigned") == 0)) {
        printf("Assign a doctor and bed before starting treatment!\n");
        return;
    }

    if (strcmp(status, "Completed") == 0) {
        for (int i = 0; i < 4; i++) {
            if (strcmp(patients[found].doctor, doctorNames[i]) == 0)
                doctorBusy[i] = 0;

            if (strcmp(patients[found].bed, bedNames[i]) == 0)
                bedBusy[i] = 0;

        }

        strcpy(patients[found].doctor, "Unassigned");

        strcpy(patients[found].bed, "Unassigned");
    }

    strcpy(patients[found].status, status);

    printf("Patient status updated successfully!\n");
}
```

```
void searchPatient() {
    char id[50];
    printf("Enter Patient ID: ");
    scanf("%49s", id);

    int found = findPatient(id);
    if (found == -1) {
        printf("Patient not found!\n");
        return;
    }

    displayPatient(patients[found]);
}

void emergencySummary() {

    int waiting = 0, treatment = 0, completed = 0;

    int critical = 0, high = 0, medium = 0, low = 0;

    for (int i = 0; i < count; i++) {
        if (strcmp(patients[i].status, "Waiting") == 0)
            waiting++;
        else if (strcmp(patients[i].status, "InTreatment") == 0)
            treatment++;
        else if (strcmp(patients[i].status, "Completed") == 0)
            completed++;

        switch (patients[i].priority) {
            case CRITICAL: critical++; break;
            case HIGH: high++; break;
            case MEDIUM: medium++; break;
            case LOW: low++; break;
        }
    }

    printf("\n===== Emergency Department Summary =====\n");
    printf("Total Patients: %d\n", count);
    printf("Waiting Patients: %d\n", waiting);
    printf("Patients in Treatment: %d\n", treatment);
    printf("Completed Treatments: %d\n", completed);
    printf("Critical Patients: %d\n", critical);
    printf("High Priority Patients: %d\n", high);
    printf("Medium Priority Patients: %d\n", medium);
    printf("Low Priority Patients: %d\n", low);
}

int main() {
```

```
int choice;

while (1) {
    printf("\n===== Smart Hospital Emergency Queue =====\n");
    printf("1. Register Patient\n");
    printf("2. Assign Emergency Priority\n");
    printf("3. Display Patients by Priority\n");
    printf("4. Call Next Patient\n");
    printf("5. Assign Doctor and Bed\n");
    printf("6. Update Patient Status\n");
    printf("7. Search Patient\n");
    printf("8. Generate Emergency Summary\n");
    printf("9. Exit\n");
    printf("Enter your choice: ");

    if (scanf("%d", &choice) != 1) {
        printf("Invalid input!\n");
        break;
    }

    switch (choice) {
        case 1:

            registerPatient();
            break;
        case 2:

            for (int i = 0; i < count; i++) {
                if (patients[i].priority >= CRITICAL &&
                    patients[i].priority <= LOW)
                    patients[i].priority = getPriority(patients[i].priority);
            }
            printf("Emergency priorities verified!\n");
            break;
        case 3:
            displayPatients();
            break;
        case 4:
            callNextPatient();
            break;
        case 5:
            assignDoctorBed();
            break;
        case 6:
            updateStatus();
            break;
        case 7:
            searchPatient();
```

```
            break;
        case 8:
            emergencySummary();
            break;

        case 9:

            printf("Exiting program...\n");
            return 0;
        default:
            printf("Invalid choice!\n");
      }

   }


   return 0;
}
```